IAM Task-01

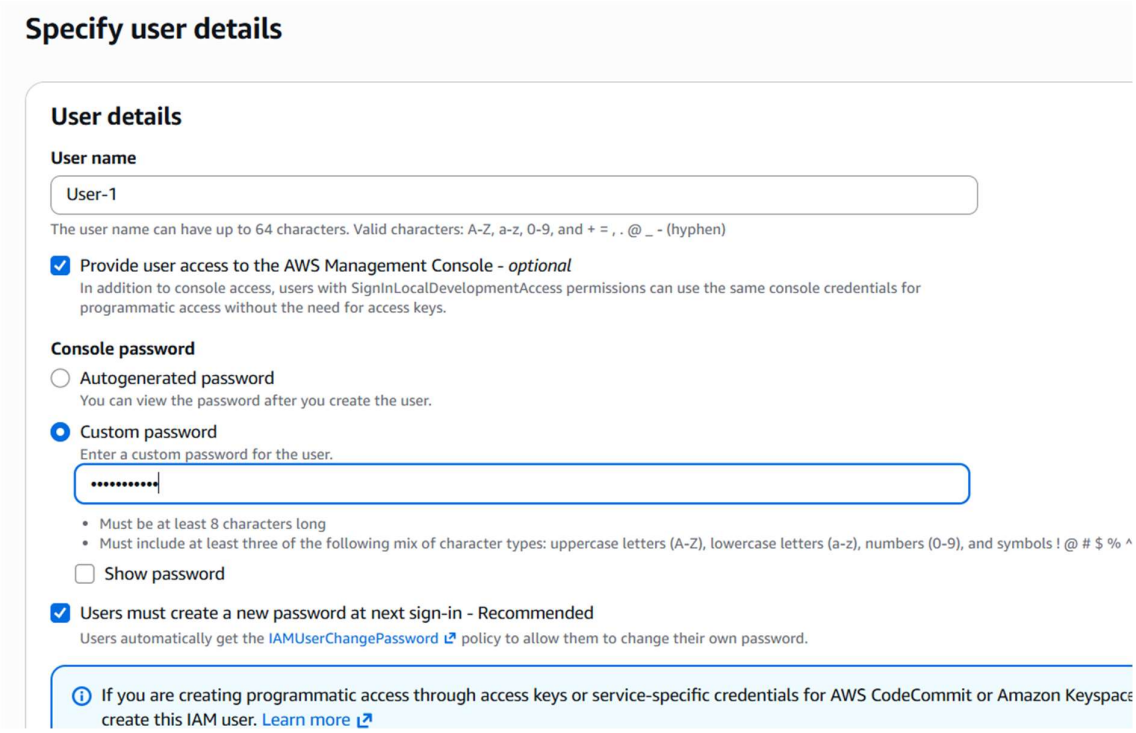
1. Create one IAM user and assign EC2 and S3 full access roles.

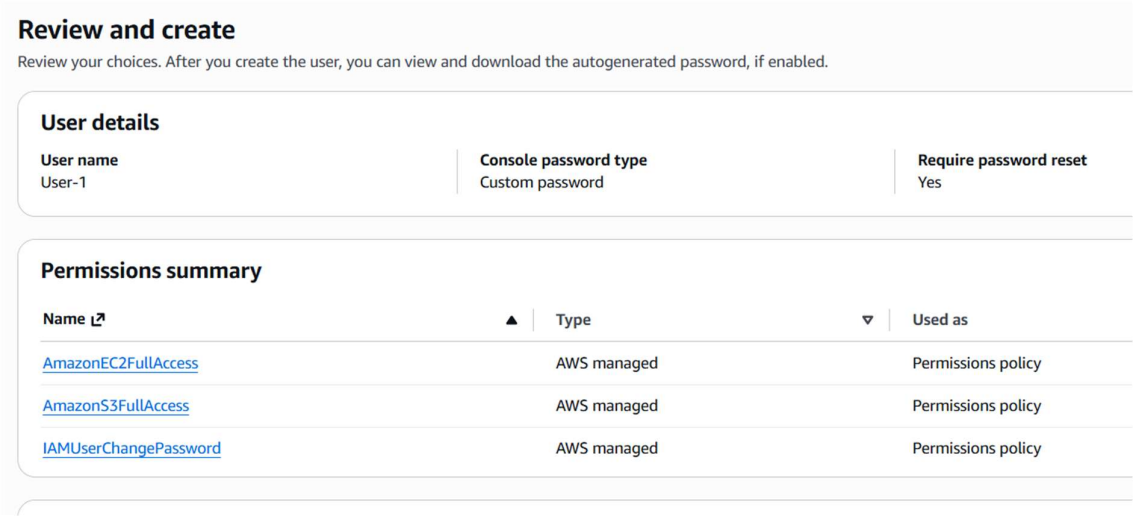
Step 1:- Go to IAM service then click on create user.

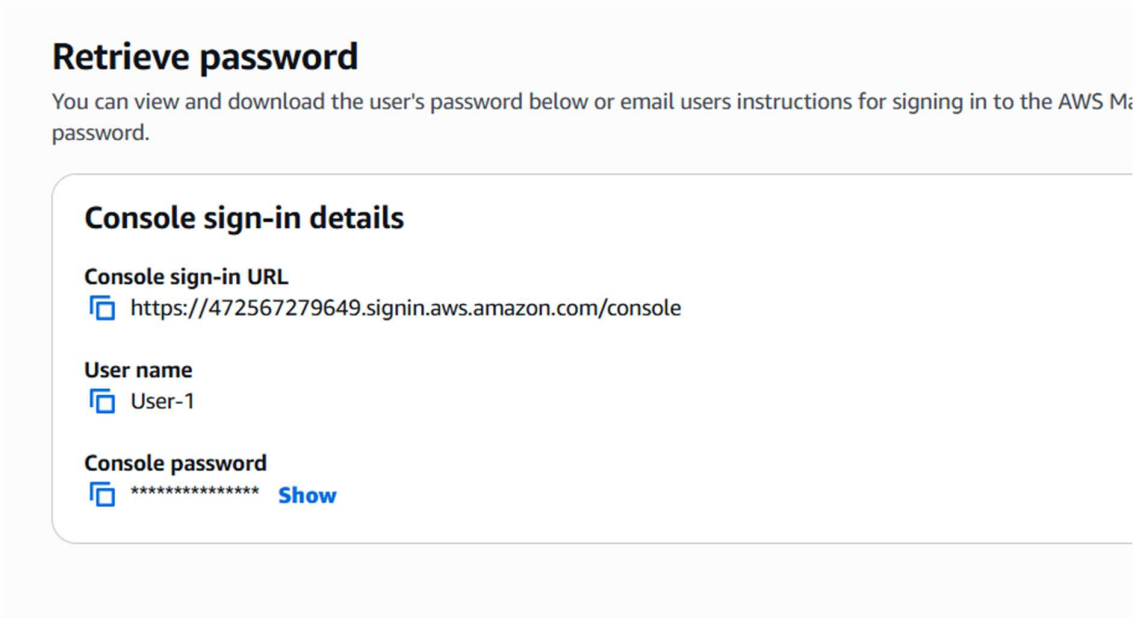
Step 2:- Allow the AWS console permission. Then click next to permission section and select amazons3fullacess and amazonec2fullaccess.

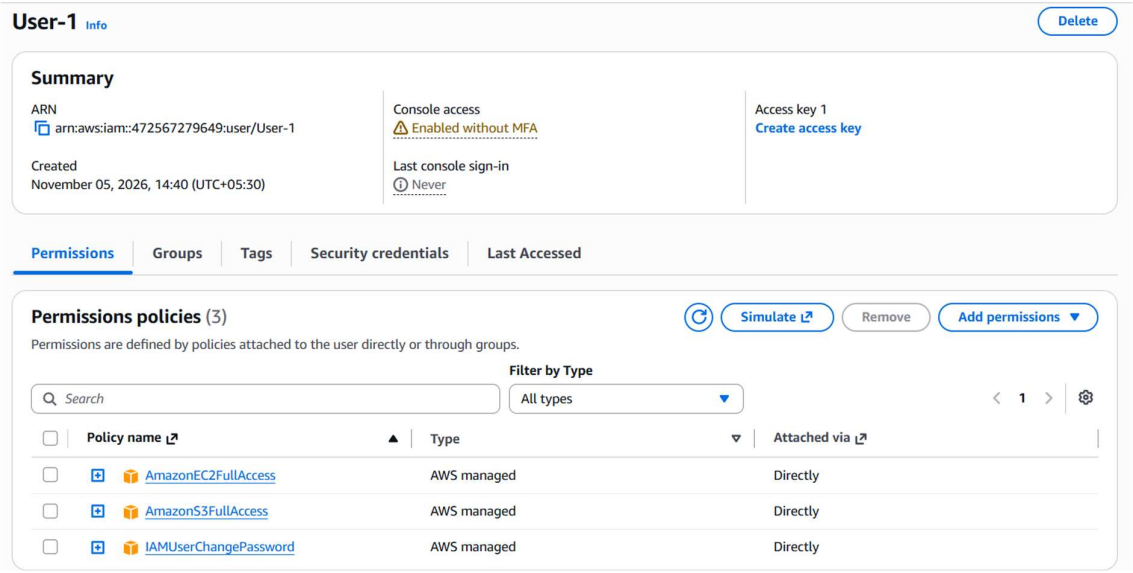
Step 3:-Create the user password so that we can also log to AWS console.

Step 4:- Now a user with EC2, S3 full access is created.







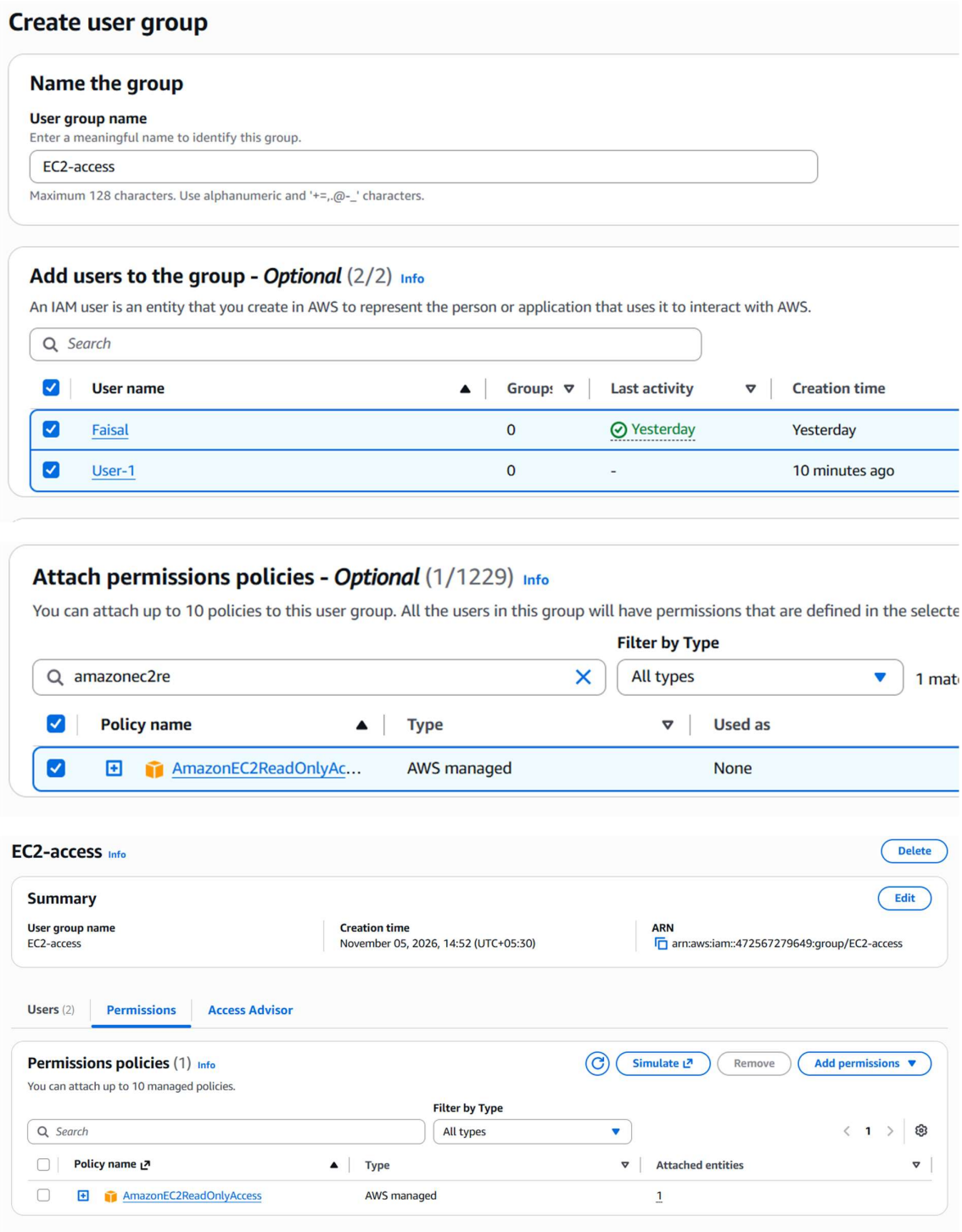


2. Create one group in IAM and assign read access for EC2.

Step 1:- Go to IAM Groups section in IAM services.

Step 2:- Then create a new group, add the users. Add EC2 read only access to the group.

Step 3:- Now the IAM group for EC2 Read access is created.



3. Create a new user named "Devops" and add to the group created in task 2.

Step 1:- Create a new user in IAM user with “Devops” as a name.

Step 2:- In permission section when creating the user select add to group option.

Step 3:- We see all the groups here. Select the group and create user.

Step 4:- Now the user is directly added to Group.

User details

User name

Devops

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☑ Provide user access to the AWS Management Console - *optional*
In addition to console access, users with SignInLocalDevelopmentAccess permissions can use the same console credentials for programmatic access without the need for access keys.

Console password

○ Autogenerated password
You can view the password after you create the user.

◉ Custom password
Enter a custom password for the user.

••••••••••

- Must be at least 8 characters long
- Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # $ % ^ & * ( ) _ + - (hyphen) = [ ] { } | '

☐ Show password

☑ Users must create a new password at next sign-in - Recommended
Users automatically get the IAMUserChangePassword policy to allow them to change their own password.

ⓘ If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. Learn more

Cancel Next

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. Learn more

Permissions options

◉ Add user to group
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

○ Copy permissions
Copy all group memberships, attached managed policies, and inline policies from an existing user.

○ Attach policies directly
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

User groups (1/1)

Create group

Search

1

☑	Group name	Users	Attached policies	Created
☑	EC2-access	2	AmazonEC2ReadOnlyAccess	2026-11-05 (3 minutes ago)

▶ **Set permissions boundary - *optional***

Cancel Previous Next

EC2-access Info

Summary

User group name	Creation time	ARN
EC2-access	November 05, 2026, 14:52 (UTC+05:30)	arn:aws:iam::472567

Users (3) | Permissions | Access Advisor

Users in this group (3)

An IAM user is an entity that you create in AWS to represent the person or application that uses it to interact with AWS.

Search

☐	User name	Groups	Last activity
☐	Devops		None
☐	Faisal		None
☐	User-1		None

4. Write a bash script to create an IAM user with VPC full access.

Step 1:- First check whether there is aws cli present or not. Then use access key and secret key of a user who has IAM full access permission.

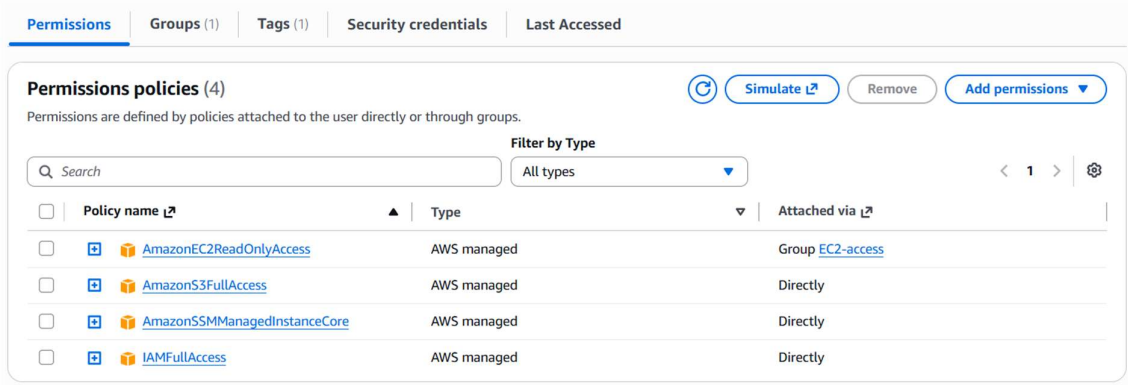
Step 2:- Then create a folder for this bash script and write the bash script. Copy the ARN from the copy for VPC full access from policy section .

Step 3:- Allow the execute permissions for the bash script. Then we can see that a user with VPC full access has been created.

```
Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
$ aws --version
aws-cli/2.39.1 Python/3.14.8 Windows/11 exe/AMD64

Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
$ aws sts get-caller-identity
{
    "UserId": "AIDAW4BZUOAQT5SUQWXLI",
    "Account": "472567279649",
    "Arn": "arn:aws:iam::472567279649:user/Faisal"
}


Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
```



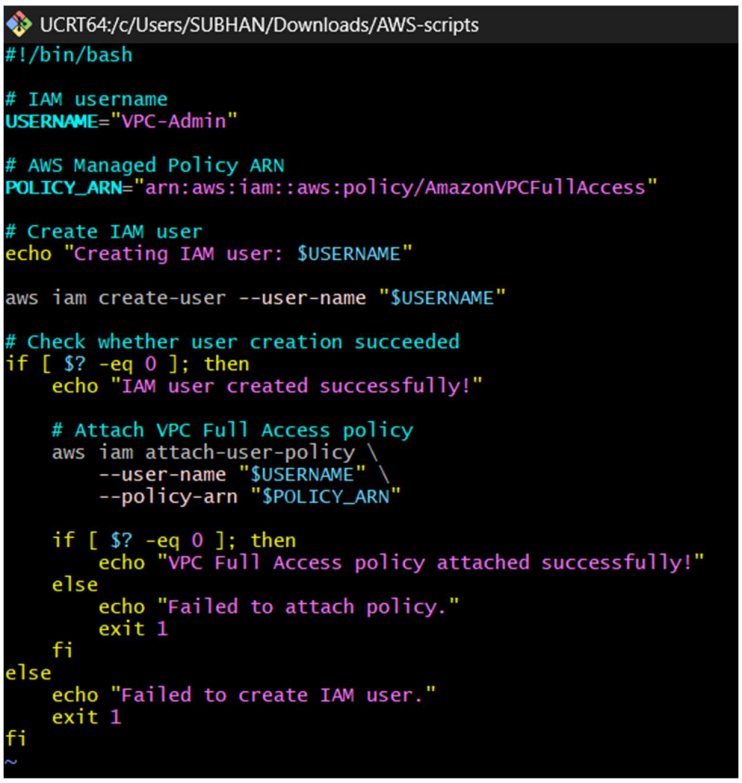

```
Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads
$ mkdir AWS-scripts

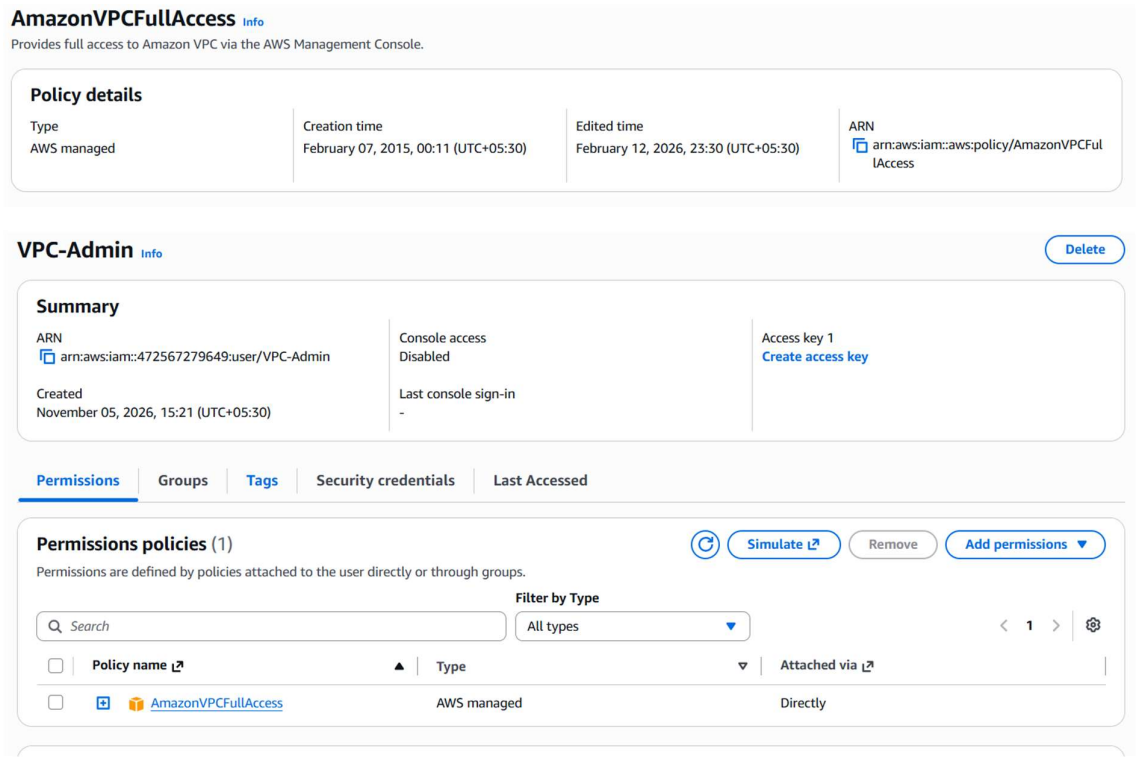
Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads
$ cd AWS-scripts/

Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
$ vi create_IAM-VPC.bash

Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
$ chmod +x create_IAM-VPC.bash

Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
```





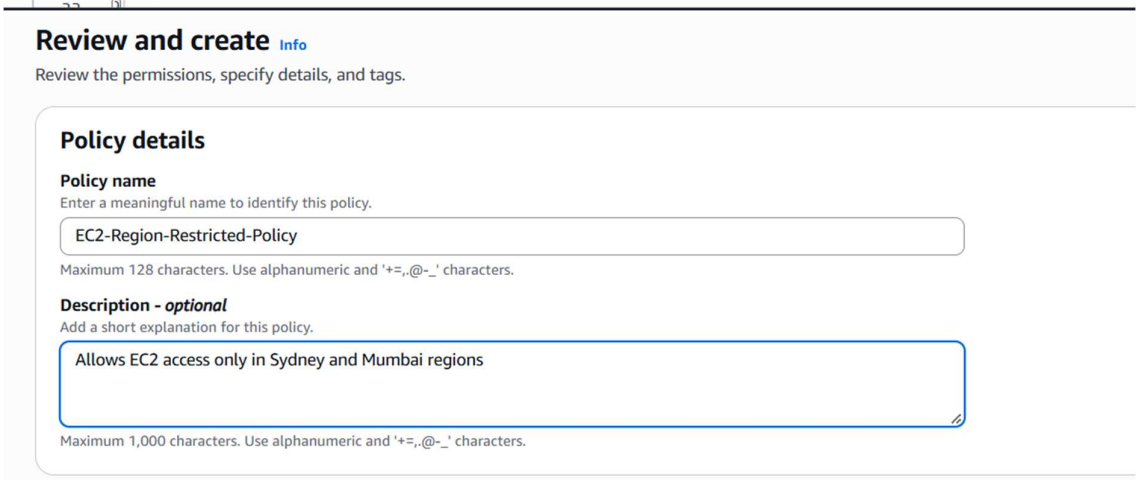
5. Create an IAM policy to allow EC2 access for a specific user in specific regions only.

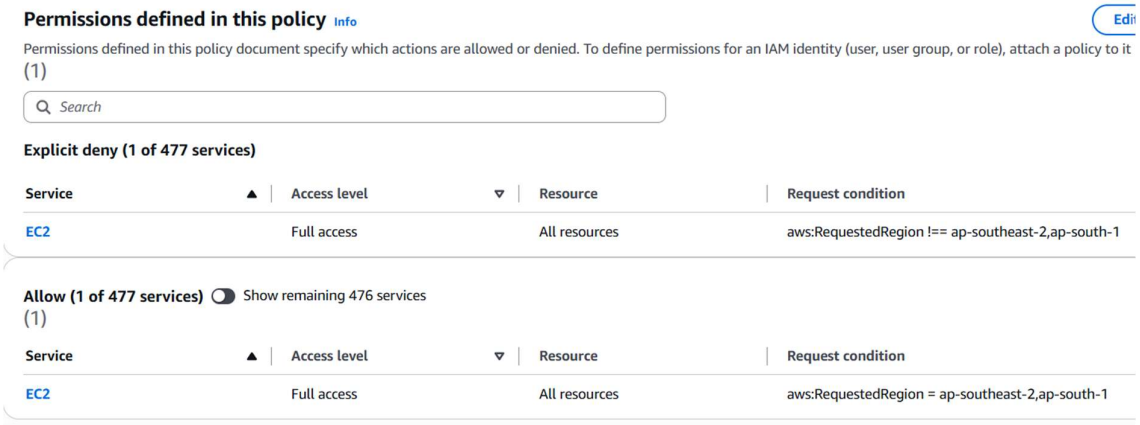
Step 1:- Go to policy and create a new policy. Then go to Json format and write the code to restrict the regions.

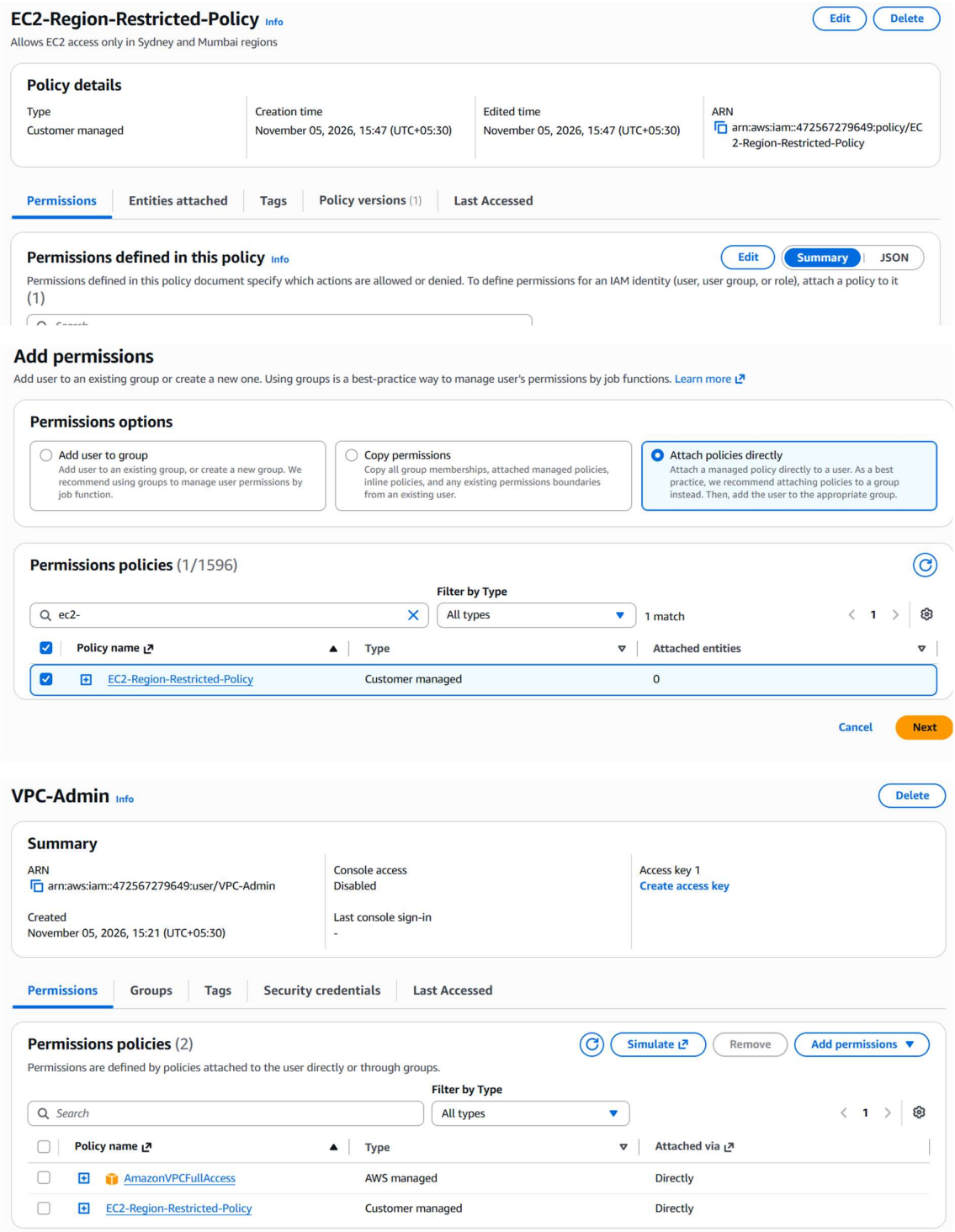
Step 2:- Now click next and give name and description for the policy.

Step 3:- After the policy is created we can go to IAM user and add permission. Then select the policy from attach policy directly assign to an individual user.







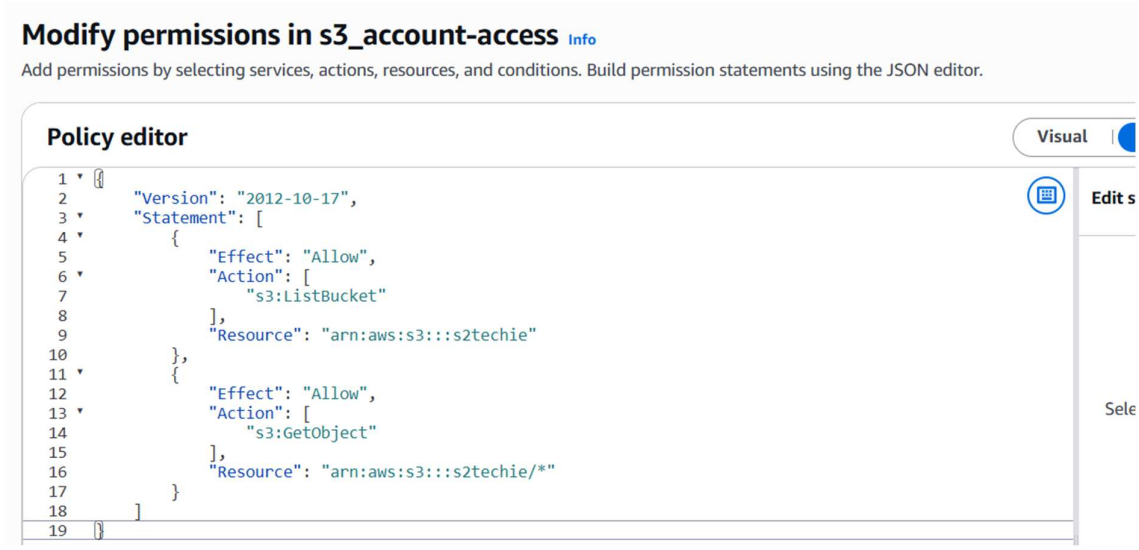


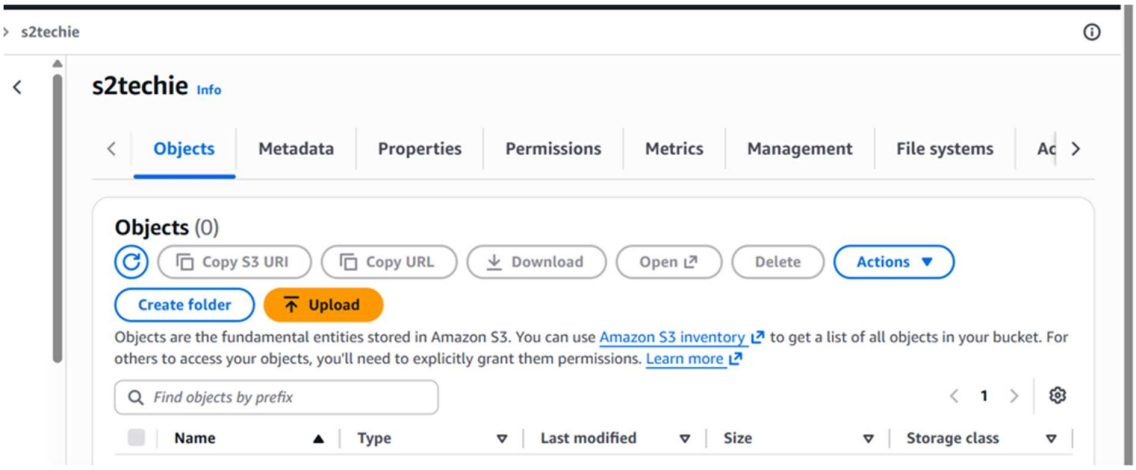
6. We have two accounts: Account A and Account B. Account A user should access an S3 bucket in Account B.

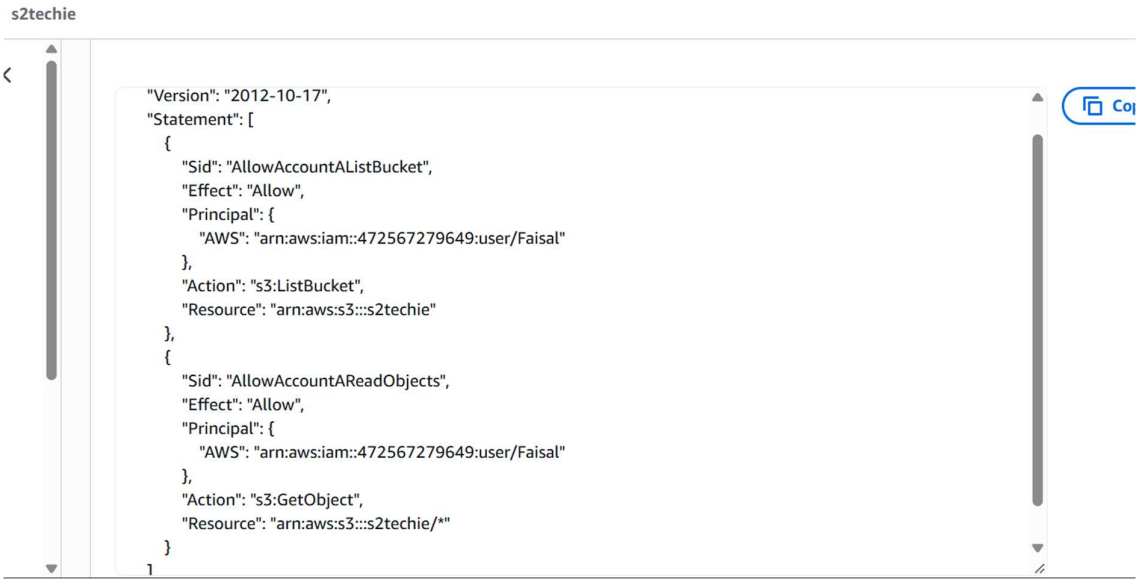
Step 1:- Go to Account A create a user with IAM full access and create a inline policy.

Step 2:- Then go to Account B create a S3 bucket, disable all public access and edit bucket policy.

Step 3:- Now open the Aws cli, then configure the IAM user. Then try to fetch the S3 bucket.







Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or
only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying an
your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. Lear

Block *all* public access

⚠ Off

▶ **Individual Block Public Access settings for this bucket**

```
Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
$ aws sts get-caller-identity
{
    "UserId": "AIDAW4BZUOAQT5SUQWXLI",
    "Account": "472567279649",
    "Arn": "arn:aws:iam::472567279649:user/Faisal"
}


Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
$ aws s3 ls s3://s2techie --region us-east-1

Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
$ aws s3 ls s3://s2techie

Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
$ aws s3 ls

Faisal@LAPTOP-4DO8SS4E UCRT64 ~/Downloads/AWS-scripts
$ aws s3api head-bucket --bucket s2techie
{
    "BucketArn": "arn:aws:s3:::s2techie",
    "BucketRegion": "us-east-1",
    "AccessPointAlias": false
}
```